

Ohanyere Francis

+234 813 432 0174 | ohanyerefrancis395@gmail.com | github.com/ohanyere | linkedin.com/in/ohanyere-francis

Professional Summary

Platform Engineer with production experience building **Internal Developer Platforms that reduce new service onboarding from 2 weeks to under 15 minutes** through self-service Golden Paths, GitOps-driven delivery, and Crossplane-backed infrastructure provisioning.

Treats platforms as internal products: **standardizing software delivery through shared CI/CD templates and progressive delivery**, automating infrastructure through declarative self-service, and **embedding shift-left security and policy governance by default** so developers inherit compliance without friction.

Strong Go engineering background with production experience building **custom Kubernetes controllers, platform orchestration APIs, and cloud-native tooling on AWS EKS**.

Core Competencies

Platform Engineering IDPs, Platform as a Product, Golden Paths, Self-Service Platforms, Platform APIs, Developer Experience (DevEx), Multi-Tenant Platforms

Cloud & Infrastructure AWS (EKS, RDS, ElastiCache, S3, SQS, DynamoDB, IAM, Route53), Kubernetes, Docker, Terraform, Helm, Crossplane

GitOps & Delivery ArgoCD (ApplicationSets, AppProjects), Argo Rollouts, GitHub Actions, Progressive Delivery, Shared CI/CD Templates, Release Engineering

Security & Governance Shift-Left Security, HashiCorp Vault, Kyverno, Cosign, Trivy, OPA/Conftest, Checkov, RBAC, OIDC, IRSA, Policy as Code

Observability Prometheus, Grafana, Loki, Alertmanager, OpenTelemetry, SLOs, Error Budgets, Runtime Visibility, Audit Trails

Backend Engineering Go, Node.js, TypeScript, REST APIs, controller-runtime, kubebuilder, GitHub API Integration, PostgreSQL, Redis

FinOps & Reliability Infracost, Cost Allocation Tags, Velero, Istio, RPO/RTO Planning, Incident Management

Professional Experience

Platform Engineer

ICT Slides – Software Engineering Incubator & Technology Training Institute

2022 – Present

Anambra State, Nigeria

- Established the organization's **platform engineering function**, owning the strategy, architecture, and evolution of an Internal Developer Platform that standardized how engineers across the organization's software engineering programs build, provision, deploy, and operate applications through self-service platform capabilities.
- Architected and evolved an **Internal Developer Platform that unified Golden Paths, reusable service templates, GitOps delivery, and self-service workflows into a standardized software delivery experience**, reducing engineer onboarding from **2 weeks to under 15 minutes** while enabling developers to independently deliver production-ready services.
- Developed **platform orchestration capabilities that abstracted repository creation, infrastructure provisioning, environment promotion, and deployment workflows behind Platform APIs**, replacing ad-hoc operational requests with standardized self-service platform operations and reducing monthly infrastructure tickets from **20+ to zero**.
- Extended Kubernetes into an **application platform through custom controllers that automated environment lifecycle management, governance, and operational controls**, reducing environment provisioning from **15 minutes and six manual kubectl operations to under 30 seconds** while standardizing engineering environments through declarative platform workflows.
- Established **self-service infrastructure capabilities that abstracted cloud resource provisioning behind declarative Kubernetes workflows**, enabling engineers to provision compliant infrastructure independently with full GitOps auditability while eliminating direct cloud console access.
- Embedded **platform governance, software supply chain security, and GitOps controls into standardized delivery workflows**, enforcing secure-by-default engineering practices while reducing deployment failures by **40%**.
- Standardized **platform observability through unified metrics, logging, deployment health, and policy visibility**, enabling faster operational diagnosis, reducing **MTTR from 2 hours to 1 hour**, and improving **MTTD by 40%**.

Key Projects

Nexus Platform – Internal Developer Platform |

nexus-idp-web.vercel.app

Next.js · Go · Node.js · Kubernetes · AWS · ArgoCD · Crossplane · Vault · Kyverno · Istio · Argo Rollouts

- Built a **self-service developer platform** that enables engineers to authenticate, create production-ready services from Golden Path templates, provision cloud infrastructure, and manage the complete software delivery lifecycle – without needing Kubernetes, ArgoCD, Helm, Crossplane, or AWS expertise. The platform handles operational complexity; developers focus on product work.

- Implemented **Golden Path service creation**: a single “Create Service” action triggers a Platform API that calls the GitHub API to generate a repository and commit standardized CI/CD workflows, Helm charts, GitOps manifests, rollout strategy, security defaults, and environment overlays automatically – developers never configure delivery infrastructure from scratch.
- Delivered a **Release Management console** exposing the full delivery lifecycle – deployment history, promotion workflows, canary rollout progress, rollback triggers, GitHub Actions CI status, and ArgoCD sync health – from a single interface, eliminating the need to context-switch across five separate tools per deployment.
- Built end-to-end **self-service infrastructure provisioning**: developers submit a resource request through the platform; the Platform API creates a Crossplane claim via a GitHub PR; ArgoCD syncs the claim; Crossplane provisions the AWS resource (PostgreSQL, Redis, S3, SQS, DynamoDB) with enforced encryption, private networking, and FinOps tags – complete GitOps audit trail, no manual cloud operations required.
- Implemented **progressive delivery** with Argo Rollouts and Istio: canary deployments route 10% of production traffic to new versions, Prometheus AnalysisTemplates evaluate error rate SLOs, and automatic rollback fires if the error rate exceeds 5% – zero-downtime deployments with data-driven promotion gates across all production services.

TeamEnvironment Operator – Custom Kubernetes Controller for Nexus Platform |
Go · controller-runtime · kubebuilder · Kubernetes · AWS EKS · Helm · ArgoCD

Live Demo

- Designed and built a **custom Kubernetes controller** in Go using controller-runtime and kubebuilder: a single **TeamEnvironment** CRD submission automatically provisions a dedicated namespace, RBAC roles and bindings, ResourceQuotas, LimitRanges, and NetworkPolicies – completing in **under 30 seconds** what previously required 6 manual `kubectl` steps and 15 minutes of platform team time.
- Implemented a **fully idempotent reconciliation loop** with exponential backoff, structured error handling, and typed status conditions – the controller safely re-reconciles on any partial failure, guaranteeing cluster state always converges to the declared spec without duplicate resources or orphaned objects.
- Built an **ArgoCD sync gate**: the operator emits a **TeamReady** status condition consumed by ArgoCD ApplicationSets as a pre-sync hook – no application deploys into a namespace until the operator confirms all RBAC, quota, and NetworkPolicy resources are provisioned and healthy.
- Packaged as a **Helm chart deployed via ArgoCD** – the operator is managed through the same GitOps pipeline it serves, with ClusterRole permissions scoped to only the resource types it manages (least-privilege operator pattern).

Education

Chukwuemeka Odumegwu Ojukwu University

B.Sc. Computer Science

Anambra State, Nigeria

ALX Africa

2022 – 2023

Software Engineering Fellowship

Certifications & Training

- **Certified Kubernetes Administrator (CKA)** – *Expected Q3 2026*
- **AWS Certified Solutions Architect – Associate** – *Expected Q4 2026*
- HashiCorp Vault Operations – Training Completed
- Istio Service Mesh Fundamentals – Training Completed